K0s

Table of Contents

The main aim of this excercise is to get familiar with Kubernetes and its ecosystem. We will be using k0s, which is a lightweight Kubernetes distribution that can be easily installed on a single machine. This makes it ideal for learning and experimentation.

Prerequisites

Before we begin, you will need to have the following software installed on your machine:

- Vagrant
- VirtualBox

To create a sandbox environment for our Kubernetes cluster, we will use Vagrant to provision a virtual machine. This allows us to easily manage and tear down our environment as needed.

```
vagrant init --box oraclelinux/10 https://oracle.github.io/vagrant-
projects/boxes/oraclelinux/10.json
# check if the box is present
vagrant box list
# export VAGRANT_VAGRANTFILE=Vagrantfile.k0s
# export VAGRANT_VAGRANTFILE=Vagrantfile.podman
vagrant up
vagrant ssh
```

Quick intro into the vagrant

Vagrant is a tool for building and managing virtual machine environments in a single workflow. It provides a simple command-line interface to create and configure lightweight, reproducible, and portable development environments. With Vagrant, you can easily create and manage virtual machines using providers such as VirtualBox, VMware, Hyper-V, and Docker.

All necessary files are present in working directory. After first run of `vagrant up` command, the vagrant will create .vagrant file, which contains the state of the machine and necessary files like as ssh keys etc.

NOTE	the project folder is automatically shared between host and guest machine, so you can edit files on your host machine and they will be available on the guest machine. (project root → /vagrant)

If you achieve a blind state of the machine, you can always run `vagrant destroy` and `vagrant up` to start from scratch.

Time to time you need save the state of the machine, before you shutdown the hypervisor machine. There is a command `vagrant halt` which will save the state of the machine and you can start it again with `vagrant up`. [vagrant-technology]

Troubleshooting

Windows have enabled hyper-v which is in collision with VirtualBox

disable hyper-v and restart the hypervisor machine. (In you case the laptop)

Windows is failing to start the virtual machine on unspecific error

Try to change value `vbox.gui` to true in the Vagrantfile and start the machine again. This will display the VirtualBox GUI when booting the machine, which may provide more information about the error.

Description of the machine

For this excercise we will be using the Oracle Linux 10 box, with preinstalled and preconfigured k0s. In the machine is present also the podman for building and pushing the docker images to the local/remote registry.

Docker

Docker is a platform for developing, shipping, and running applications in containers. Containers are lightweight, portable, and self-sufficient units that can run consistently across different environments. Docker allows you to package your application and its dependencies into a container image, which can then be run on any machine that has Docker installed.

The base unit of the docker is `dockerfile`, which is a text file that contains instructions for building a docker image. The dockerfile specifies the base image to use, the commands to run, and the files to include in the image.

The main advantage of using docker is that it allows you to create and manage your application in a consistent and reproducible way. It means you don't have to worry about the conflicts between different versions of libraries and dependencies on your machine, because everything is packaged in the container. It also prevents problems like as:

- CPU resource
- Memory resource
- Networking (port conflicts)
- File system (conflicts between different versions of files)
- etc.

NOTE	In one folder we can have a multiple dockerfiles, but we need to specify the name of the dockerfile when building the image.

Docker exec

The `docker exec` command allows you to run a command in a running container. This is useful for debugging and troubleshooting your application, as well as for running administrative tasks inside the container. For example, you can use `docker exec` to open a shell inside the container, or to run a command that is not part of the container's main process.

IMPORTANT	The exec is the operation which is not persistent during the restart or recreation of the container. From that reason is highly recommended to use it only for debugging and troubleshooting purposes, and not for running the main process of the container.
IMPORTANT	The exec command should be applied only on running containers, otherwise it will fail with error "No such container". If you want to run a command in a

> stopped container, you can use `docker run` command with the `--rm` flag, which will remove the container after the command is executed.

Store of images

There are several options for storing and sharing docker images:

- Docker Hub: This is the default public registry for docker images. It allows you to store and share your images with the community, and also to find and use images created by others.
- Private registry: You can set up your own private registry to store and share your images within your organization. This can be done using tools like Harbor, Nexus, or Artifactory.
- Local registry: You can also run a local registry on your machine to store and share images within your local network. This can be done using the `registry` image from Docker Hub.
- Container tarball: You can also save your docker images as tarballs and share them manually. This can be done using the `docker save` and `docker load` commands.

Examples

1 run empty container

```
podman run -it --rm alpine:3.22.4 sh
```

2 build my own image

```
podman build -t my-appache2 -f Dockerfile.hello .
```

3 run my own image

```
podman run -p 8333:80 -dit --rm localhost/my-apache2:latest
```

4 change default page

```
podman run -p 8333:80 -v  -dit --rm localhost/my-apache2:latest
```

5 dynamically shared content

```
podman run -p 8333:80 -v ./content:/var/www/localhost/htdocs -dit --rm localhost/my-
apache2-extend:latest
```

6 export and import image

```
podman save -o my-apache2.tar localhost/my-apache2:latest
podman ps -a
podman rm <container_id>
podman image rm localhost/my-apache2:latest
```

```
podman load -i my-apache2.tar
```

Summary

In this section, we have covered the basics of Docker, including what it is, how it works, and how to use it to build and run containerized applications. We have also discussed the different options for storing and sharing docker images, and provided some examples of common docker commands.

From provided examples we can imaggine how to `lift and shift` our application to the containerized environment, which is the first step towards deploying it on Kubernetes.

Troubleshooting

Check the container health and interact with it

```
podman ps -a
podman logs <container_id>
podman inspect <container_id>
podman exec -it <container_id> sh
```

Check the image and its layers

```
podman images
podman history <image_name>
```

Remove the container and image

```
podman ps -a
podman rm <container_id>
podman images
podman rmi <image_name>
```

Check the network and port mapping

```
podman ps -a
podman port <container_id>
podman inspect <container_id> | grep -i "ipaddress"
```

How to upload container into the registry

```
podman login <registry_url>
podman tag <image_name> <registry_url>/<image_name>:<tag>
```

```
podman push <registry_url>/<image_name>:<tag>
```

How to pull container from the registry

```
podman login <registry_url>
podman pull <registry_url>/<image_name>:<tag>
```

Kubernetes

Kubernetes is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications. It provides a powerful and flexible framework for running applications in a distributed environment, and is widely used in production environments around the world. Kubernetes allows you to manage your applications in a consistent and reproducible way, and provides features such as automatic scaling, self-healing, and rolling updates.

The cluster are synchronised on port 6443, which is the default port for the Kubernetes API server. This means that you can use kubectl to interact with the cluster and manage your applications.

There are several types of nodes in a cluster:

- Control plane nodes: These nodes run the Kubernetes control plane components, such as the API server, scheduler, and controller manager. They are responsible for managing the cluster and ensuring that the desired state of the applications is maintained.
- Worker nodes: These nodes run the application workloads, and are managed by the control plane. They run the kubelet and kube-proxy components, which are responsible for managing the containers and networking on the worker nodes.
- Infrastructure nodes: These nodes are optional and run the infrastructure components, such as the container runtime, logging, and monitoring. They are responsible for providing the necessary resources and services for the cluster to function properly.

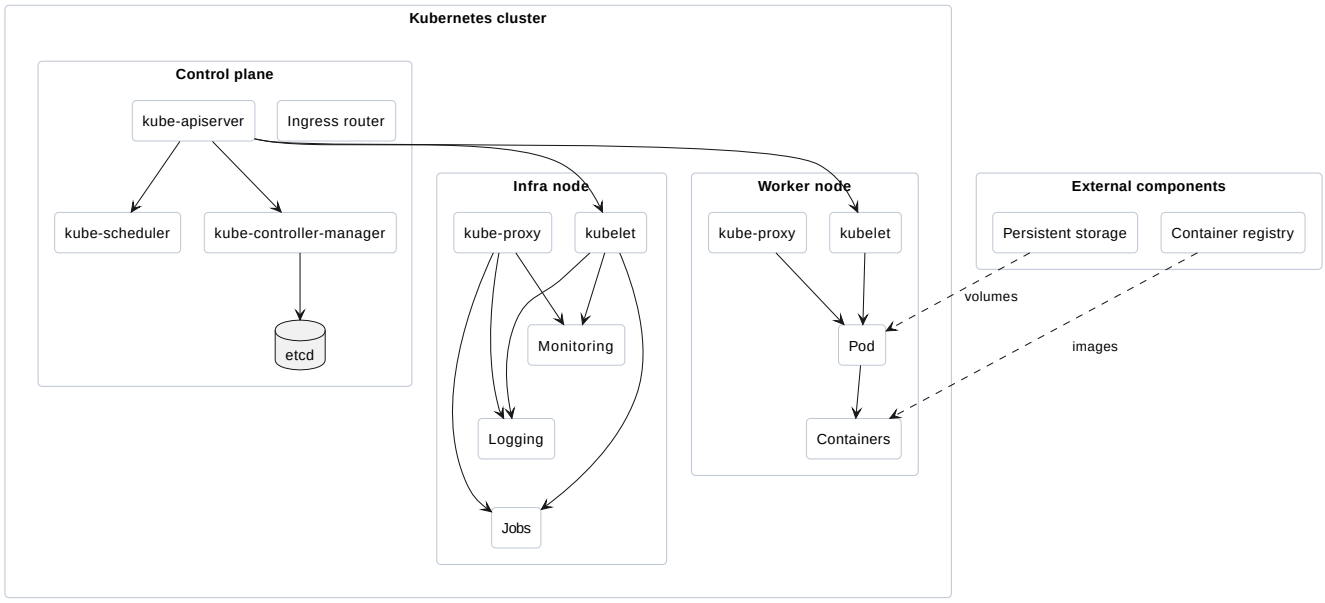

Figure 1. Example of Kubernetes cluster architecture

So if we imagine the communication between the actor and the cluster, we can see that the actor facing ingress router, which is responsible for routing the incoming traffic to the appropriate services in the cluster. There we can can imagine that the ingress recognises the request with suburi for example `/hello` and routes it to the different service, which is responsible for hangling the request and forwarding it to the appropriate pods.

The simple communication we can imagine at the following figure.

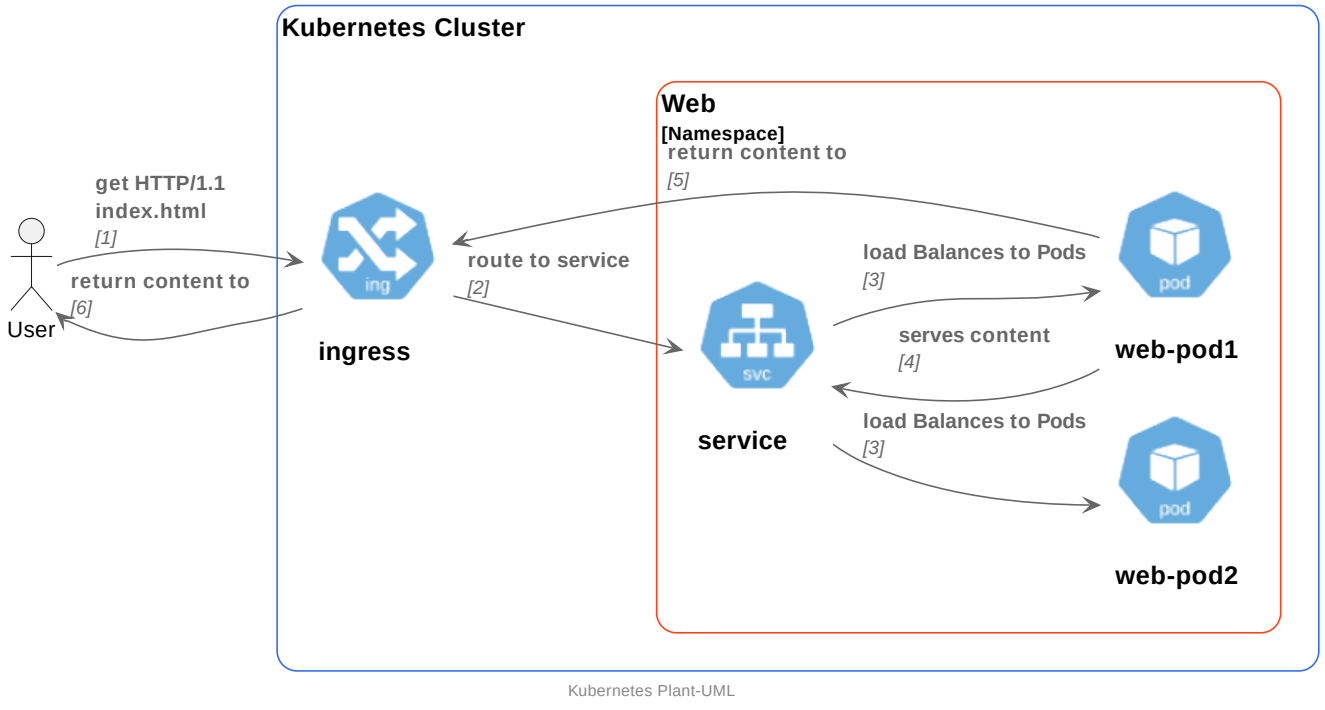

Figure 2. Example of communication in the cluster

All objects in the cluster are driven by labels, which are key-value pairs that are attached to the objects. Labels are used to organize and select subsets of objects, and can be used for a variety of purposes, such as grouping related objects together, or selecting objects based on their characteristics. For example, you can use labels to select all pods that belong to a particular application, or to select all nodes that have a particular hardware configuration.

Nice example of the labels are in folder `app` where we have deployment and service for the hello world application. There you don't have any IPs or ports, but you have labels, which are used to select the appropriate pods and services. Instead of using direct port mapping/rounting you just expose the ports (Tell the DevOps that the application is listening on port).

Type of deployments

There are several types of deployments in Kubernetes, each with its own use case and characteristics:

- **Deployment**: This is the most common type of deployment, and is used for stateless applications. It provides features such as rolling updates, self-healing, and scaling.
- **StatefulSet**: This type of deployment is used for stateful applications, such as databases. It provides features such as stable network identities, persistent storage, and ordered deployment and scaling.
- **DaemonSet**: This type of deployment is used for running a copy of a pod on every node in the cluster. It is often used for running system-level services, such as log collectors or monitoring agents.
- **Job**: This type of deployment is used for running a batch job or a one-time task. It provides features such as parallelism and retries, and is often used for running tasks that need to be completed within a certain time frame, such as data processing or backups.
- **CronJob**: This type of deployment is used for running a job on a schedule. It provides features such as cron-like scheduling and time-based execution, and is often used for running tasks that need to be executed at specific times, such as periodic data processing or maintenance tasks.
- **Custom resources**: Kubernetes also allows you to define your own custom resources, which can be used to represent any type of object in the cluster. This allows you to extend the functionality of Kubernetes and create your own custom controllers and operators to manage your applications.

How the cluster recognises the state of the application

Kubernetes uses a declarative approach to managing applications, which means that you specify the desired state of your application in a manifest file, and Kubernetes will ensure that the actual state of the application matches the desired state. This is done through a process called reconciliation, where Kubernetes continuously monitors the state of the application and takes action to bring it back to the desired state if it deviates. For example, if you specify that you want to have 3 replicas of a pod running, and one of the pods crashes, Kubernetes will automatically create a new pod to replace the crashed one, ensuring that the desired state of 3 replicas is maintained.

Each container haves a `liveness` and `readiness` probe, which are used to check the health of the container. The liveness probe is used to check if the container is alive and running, while the readiness probe is used to check if the container is ready to accept traffic. If the liveness probe fails, Kubernetes will restart the container, while if the readiness probe fails, Kubernetes will stop sending traffic to the container until it becomes ready again.

The `startup` probe is used to check if the container has started successfully. If the startup probe

fails, Kubernetes will consider the container as failed and will not attempt to restart it. This is useful for applications that have a long startup time, as it allows Kubernetes to wait for the application to start before considering it as failed.

probes documentation

pod vs. sidecar

A pod is the basic unit of deployment in Kubernetes, and is used to run one or more containers that are tightly coupled and share the same network namespace. A sidecar is a container that runs alongside the main application container in a pod, and is used to provide additional functionality or services to the main application. For example, a sidecar can be used to provide logging, monitoring, or proxying services to the main application container. The sidecar pattern allows you to separate concerns and modularize your application, making it easier to manage and maintain.

secrets

secrets

configmaps

configmaps

Summary

In this section, we have covered the basics of Kubernetes, including what it is, how it works, and how to use it to deploy and manage containerized applications. We have also discussed the different types of deployments in Kubernetes, and how Kubernetes uses a declarative approach to managing applications through reconciliation. We have also covered the concepts of liveness, readiness, and startup probes, which are used to check the health of the containers and ensure that the desired state of the application is maintained.

Examples

1 get state of the cluster

```
kubectl get nodes
kubectl get namespaces
kubectl get all -A
kubectl get all -n <namespace>
kubectl describe <object_type>/<object_name> -n <namespace>
```

2 deploy application

```
kubectl apply -f deployment.yml -n <namespace>
```

3 remove stuffs

```
kubectl delete -f deployment.yml -n <namespace>
```

4 import docker image into the cluster

```
/usr/local/bin/k0s ctr -n k8s.io images import my-deployment-images.tar
/usr/local/bin/k0s ctr -n k8s.io images ls
```

Troubleshooting

The Kubernetes is not working properly

```
systemctl status k0scontroller
journalctl -u k0scontroller -f
systemctl status firewalld
journalctl -u firewalld -f
```

Kubernetes have a problem with the network

Deploy a container with right toolset like as:

- traceroute
- ping
- nmap
- nslookup
- tcpdump
- etc.

Helm

Helm is a package manager for Kubernetes that allows you to easily deploy and manage applications in a Kubernetes cluster. It provides a simple and consistent way to package, deploy, and manage applications, and is widely used in production environments around the world. Helm uses a templating system to define the desired state of your application, and provides features such as versioning, dependency management, and rollbacks.

Helm charts

```
my-chart/
 ├──── .helmignore          # Patterns to ignore when packaging the chart
 ├──── Chart.yaml           # Metadata about the chart (name, version, API version)
```

```
├──── values.yaml           # Default values for template variables
├──── charts/               # Directory containing any external charts (subcharts)
├──── templates/            # Directory for Kubernetes manifest templates
│     ├──── _helpers.tpl     # Partial templates and reusable helper functions
│     ├──── deployment.yaml  # Kubernetes Deployment manifest
│     ├──── service.yaml     # Kubernetes Service manifest
│     ├──── ingress.yaml     # Kubernetes Ingress manifest
│     └──── NOTES.txt        # Post-installation instructions displayed to the user
└──── README.md             # Human-readable documentation for the chart
```

The nice example is in the folder helm where is a simple chart for our test application.

Examples

1 deploy application with helm

```
/usr/local/bin/helm install hello-kubernetes ./helm
```

2 remove application with helm

```
/usr/local/bin/helm uninstall hello-kubernetes
```

3 check the state of the application

```
kubectl get all -n frantisek-hello
/usr/local/bin/helm install hello-kubernetes ./helm --dry-run
/usr/local/bin/helm status hello-kubernetes
/usr/local/bin/helm history hello-kubernetes
```

4 rollback to previous version

```
/usr/local/bin/helm rollback hello-kubernetes 1
```

Extending the knowledge

In the deployment world you can have a lot of different approaches and tools, which can be used to deploy and manage the applications in the cluster. One of the most popular tools is ArgoCD, which is a declarative, GitOps continuous delivery tool for Kubernetes. It allows you to manage your applications in a Git repository, and automatically deploys them to the cluster when changes are made. ArgoCD provides features such as automatic rollbacks, health checks, and multi-cluster support.

Summary

For delivery package it will be necessary to create a tarball with exported docker images and helms

charts, which will be used to deploy the application in the cluster.

IMPORTANT	The HELM charts must contain pullpolicy set to `IfNotPresent`, which means that the cluster will try to pull the image from the local registry first, and if it is not present in a case of isolated datacenter.

Reference

- [k0s] https://docs.k0sproject.io/v1.21.2+k0s.1/install/
- [k0s-network-fw] https://docs.k0sproject.io/v1.35.3+k0s.0/networking/ https://github.com/Voronenko/k0s-mini
- [cheatcheat-helm] https://helm.sh/docs/intro/CheatSheet
- [kubernetes-quick-reference] https://kubernetes.io/docs/reference/kubectl/quick-reference/
- [kube-hello-world] https://hub.docker.com/layers/paulbouwer/hello-kubernetes/1.10.1/images/sha256-2ad94733189b30844049caed7e17711bf11ed9d1116eaf10000586081451690b
- [kube-plantuml] https://github.com/dcasati/kubernetes-PlantUML
- [dockerfile-reference] https://docs.docker.com/reference/dockerfile/
- [dockerhub-alpine] https://hub.docker.com/_/alpine/tags
- [vagrant-technology] https://developer.hashicorp.com/vagrant